

Adversarial Maze Navigation: Comparing Search Strategies Under Pursuit, Dynamic Environments, and Partial Observability

By: Namya Singh & Vikramaditya Sogani

Abstract

This paper describes a maze navigation game where an AI agent tries to reach the goal while being chased by a pursuer. We built and tested nine different agent strategies, including game tree search methods like Minimax, Alpha-Beta, and Expectimax, a real time learner called LRTA*, local search methods like Hill Climbing and Beam Search, and three agents that work with limited vision like Online A*, Online LRTA*, and an Exploration agent. The maze can be static or have walls that shift during the game. We also tested a fog of war mode where the agent cannot see the full map. We ran many experiments across different maze sizes, wall densities, and how far the agent can see, and found that Alpha-Beta and Minimax won about 40.6% of games when the agent could see everything, while Online A* was the best in fog of war mode at 43.2%. The project also comes with a visual game you can watch or play, a tool to run many experiments at once, and charts that show the results.

1. Introduction

Getting from one place to another while someone is trying to stop you is one of the oldest problems in AI research. It shows up in robotics, self-driving cars, video games, and even security systems. The maze problem, where an agent needs to find a path from a start point to a goal through a grid of walls, has been used to test search algorithms for decades. Things get much more interesting when you add a pursuer that is actively trying to catch the agent: now the agent cannot just find any path to the goal, it has to find one fast enough to stay ahead.

Most earlier work on grid pathfinding looks at one algorithm at a time in a simple, fully visible maze. Yap (2002) ran a careful study of how the shape of the grid affects how well A* and IDA* work, comparing square grids, grids with diagonal moves, and hexagonal grids. His results show that the type of grid you use has a huge effect on how many steps these algorithms need to explore. Our project uses the same kind of square four-direction grid that Yap treats as the standard case, but we ask a different question: instead of changing the grid, we change the algorithm. We tested nine different decision-making strategies in the same environment to see how they compare.

We wanted to understand how different AI approaches hold up when things get harder. The agent in our system faces a pursuer that can range from completely random to a smart A* tracker. We test four difficulty levels: full visibility with a fixed maze, full visibility with walls that randomly shift, fog of war with a fixed maze, and fog of war with shifting walls. In each setting we compare agents that think very differently about how to make a decision.

The main things we built and studied are a single shared environment that lets us compare all nine agents fairly, a breakdown of the tradeoffs between how smart an agent is versus how fast it runs and how often it wins, and a live visual game where you can watch any algorithm play in real time.

1.1 Background and Related Work

The two algorithms most central to our grid setting are A* and IDA*, both studied by Yap (2002). A* scores each possible move using a formula $f(n) = g(n) + h(n)$, where $g(n)$ is the distance already traveled and $h(n)$ is a guess of how far the goal still is. As long as that guess never overpredicts the real distance, A* is guaranteed to find the shortest path. The downside is that it has to remember every state it has visited, which uses a lot of memory. IDA* fixes that by running a series of depth searches with a growing cost limit. It uses almost no memory and still finds optimal paths, but it has to revisit many states along the way.

Yap's work helps explain our choice of the Manhattan distance as our main guide. He showed that a tighter guess directly reduces how many cells an algorithm has to look at, and that the Manhattan distance is the right choice for four-direction square grids. He also showed that the search tree for IDA* on a four-direction grid grows at roughly $O(3.00 \text{ to the power of depth minus heuristic})$, which gives us a baseline for thinking about how our algorithms scale when mazes get bigger or denser.

Our project goes well beyond what Yap studied. His work only looks at A* and IDA* in a single-agent, fully visible, static setting. We add a pursuer that is actively trying to catch the agent, walls that can change position during the game, and a fog of war mode where the agent can only see a small area around itself. We also include uninformed methods like BFS and DFS, local search methods like Hill Climbing and Beam Search, full game tree search methods like Minimax and Alpha-Beta, and three agents built specifically for limited(partial) visibility. Our goal is to see how all of these approaches hold up in one shared, challenging environment.

2. Methods

2.1 Environment Design

The maze is a grid of cells where each cell can be open, a wall, or one of several special types. Trap cells freeze the agent for three turns when stepped on. Power-up cells give the agent a temporary ability. Mud cells cost three steps to cross, water cells cost two, and road cells look different but cost the same as a regular open cell. There are four types of power-ups: one that briefly lifts the fog of war so the agent can see everything, one that lets the agent take two steps per turn, one that protects the agent from traps, and one that freezes the pursuer in place.

The static maze is generated randomly using a configurable size and how many walls you want. The dynamic version adds wall shifts that happen at set intervals, usually every six steps. This means any agent that makes a full plan at the start and then just follows it will often find that the plan no longer works. Algorithms like LRTA* and the online agents handle this naturally because they only plan one step ahead at a time.

The game takes turns: the agent moves first, then the pursuer. The agent wins by reaching the goal cell. The pursuer wins by landing on the same cell as the agent. A step limit is imposed to prevent infinite games; in our experiment pipeline this is 100 steps for full-observability runs and 200 steps for partial-observability runs.

Starting screen of the visual simulator.



2.2 Full Visibility Agents

When the agent can see everything, it knows the full maze layout, where the pursuer is, and where the goal is. We built six agents for this mode.

Minimax treats the game as a two-player competition. The agent tries to maximize its chances and the pursuer tries to minimize them. We cap the search at four moves ahead because looking further would take too long. At the end of the search tree, positions are scored: the agent winning is worth +1000, getting caught is worth -1000, and running out of time is 0. Positions in the middle of the tree are scored based on how close the agent is to the goal, how close the pursuer is to the goal, and how far apart the two players are.

Alpha-Beta is Minimax with a shortcut. It skips any branch of the search tree that cannot possibly change the final answer. Because it searches smarter, we let it look six moves ahead instead of four, and it still runs faster than Minimax.

Expectimax works like Minimax except it does not assume the pursuer plays perfectly. Instead, the pursuer is modeled as somewhat random: it moves toward the agent most of the time but sometimes takes a random step. We used a 50% randomness setting, which models a pursuer that is unpredictable half the time. This makes Expectimax more realistic for pursuers that are not perfectly smart.

LRTA* is a real time search method. It makes exactly one move per step and keeps a table of learned scores for each position it has visited. After each move it updates the score for the cell it just left so the agent does not loop back there again. Because it never plans more than one step ahead, it is extremely fast and naturally handles walls that shift, since it never makes a plan it would have to throw away.

Hill Climbing always moves to whichever neighboring cell looks closest to the goal. It is fast but can get stuck when all the nearby cells are worse than the current one, which happens often in mazes. Once stuck, it has no way to backtrack, which is a big problem when the pursuer is approaching.

Beam Search keeps a small group of candidate positions open at each step, fixed at three, and only keeps the best ones by how close they are to the goal. It avoids some of the traps that Hill Climbing falls into, but like Hill Climbing it can still miss the goal if the best path requires going through cells that look bad along the way.

2.3 Fog of War Agents

In fog of war mode, the agent starts with a completely blank map and gradually learns what is around it as it moves. At each step it can see all cells within a certain distance. Unknown cells are

assumed to be open when planning, which can lead the agent into dead ends it did not know about. We built three agents for this mode.

The Online A* agent replans a full path every single step based on what it currently knows about the map. If it turns out a cell it planned to walk through is actually a wall, it just replans at the next step. This makes it complete on the known map and always following the best path given current information, but it does do more computation than the other fog of war agents.

The Online LRTA* agent is the fog of war version of LRTA*. It keeps the same learned score table but applies it to the partial map. Unknown cells start with a score based on their straight-line distance to the goal. If a cell is discovered to be a wall, its score is set to infinity so the agent will not try to go there. This agent never runs a full search, just one step at a time.

The Exploration agent works in two phases. In the first phase it navigates toward the nearest frontier, which is a known open cell that borders at least one unknown cell. It systematically builds up a map of the maze this way. Once it can see the goal in its map, it switches to the second phase and runs A* on the now-known map to go straight there. This models a robot that maps first and navigates after.

2.4 Pursuer Strategies

We built four pursuer types. The random pursuer picks any legal move at random and serves as the easiest difficulty. The greedy pursuer always moves to whichever neighbor brings it closest to the agent, making it a direct and predictable chaser. The beam pursuer uses a beam search toward the agent, so it can navigate around walls while staying computationally fast. The A* pursuer runs a full A* search to the agent every single turn, making it the hardest to escape.

2.5 Experiment Setup

We ran experiments across two visibility modes (full and fog of war), two maze types (static and shifting walls), two maze sizes (11x11, 13x13), two wall densities (15% and 25%), and three fog of war radii (2, 4, and 6 cells). Each setup was repeated with two different random mazes, and all four pursuer strategies were tested in every setup. We recorded win rate, average number of steps to finish, average time per game in milliseconds, how many cells the algorithm explored, and for fog of war agents, what fraction of the map was revealed. Results were saved to CSV files and turned into charts automatically.

3. Results

3.1 Full Visibility Results

The Following table shows the overall results for full visibility experiments, averaged across all maze sizes, wall densities, and pursuer types.

Example maze during gameplay showing the agent, pursuer, and terrain types.



Full Visibility Agent Performance Summary

Agent	Win Rate	Avg Steps	Avg Runtime (ms)	Avg Nodes Expanded
Alpha-Beta	40.6%	34.7	9.81	5,070
Minimax	40.6%	34.7	18.38	10,772
Expectimax	40.6%	35.9	24.47	11,097
LRTA*	37.5%	15.0	0.09	0
Beam Search	39.1%	14.2	1.67	428
Hill Climbing	21.9%	21.7	0.71	163

Alpha-Beta, Minimax, and Expectimax all won about 40.6% of games, showing that all three game tree methods perform about the same when given enough depth to think ahead. Alpha-Beta got there in about half the time of Minimax by skipping parts of the search that could not change the answer. Expectimax was the slowest of the three because it has to calculate probabilities across all the pursuer moves instead of just finding one best response.

LRTA* won 37.5% of games but finished in under 0.1 milliseconds, making it by far the fastest algorithm tested. Compared to Minimax, which averaged 18 milliseconds, LRTA* was over 200 times faster. It gives up a little win rate in exchange for extreme speed. Beam Search landed in the middle at 39.1% win rate and 1.67 ms, which is a reasonable balance between smart decision-making and fast execution.

Hill Climbing came in last at 21.9% win rate. The reason is that mazes have many spots where all nearby cells look worse by the straight-line distance measure, even though the real path requires moving through them. When the agent gets stuck at one of these spots, the pursuer keeps closing in and the game is usually lost.

The game tree agents were most affected by what type of pursuer they faced. Against a random pursuer, they sometimes lost because the random movement was unpredictable and could cut off paths. Against a greedy pursuer, they did better because the search tree can model predictable behavior well. Against an A* pursuer, win rates dropped across the board because the pursuer navigated around walls efficiently and gave the agent fewer escape routes.

3.2 Fog of War Results

Table 2 shows the results for fog of war mode, averaged across all visibility radii, maze sizes, wall densities, and pursuer types.

Fog of War Agent Performance Summary

Agent	Win Rate	Avg Steps	Avg Runtime (ms)	Avg Cells Explored
Online A*	43.2%	14.2	2.81	54.1%
Online LRTA*	32.8%	15.0	0.09	51.1%
Exploration	31.2%	19.4	0.29	59.4%

Online A* won 43.2% of games, which is actually higher than the best full visibility agents. This might seem surprising, but it makes sense when you think about it. The fog of war experiments included a wider range of setups, and Online A* benefits from replanning every step because it always responds to the most current information it has. By assuming unknown cells are open and replanning whenever that turns out to be wrong, it often finds routes through unexplored areas faster than agents that already know every wall.

Online LRTA* won 32.8% and ran in 0.09 milliseconds, matching the speed of the full visibility LRTA*. It falls behind Online A* mainly because its learned score table can sometimes steer it toward dead ends that looked fine when the scores were first set.

The Exploration agent won 31.2% even though it explored more of the map than the other two agents (about 59% on average). It loses more often because the first phase of its strategy involves navigating toward the edges of the known area rather than toward the goal. During that time the pursuer keeps moving, which means the agent is spending steps mapping instead of escaping. In

very dense mazes where the goal could be anywhere, this tradeoff pays off more, but on average the time cost of mapping is too high.

How far the agent can see matters. When the visibility radius is only 2 cells, agents bump into walls they did not know about and have to replan often. When the radius is 6, they can see far enough ahead to make better decisions. The Exploration agent benefits most from a larger radius because it finds new frontier areas sooner and wastes fewer steps doubling back.

3.3 Shifting Wall Results

When walls shift during the game, all full visibility agents actually performed better on dynamic mazes than static ones. Alpha-Beta, Minimax, and Hill Climbing each gained 12.5 percentage points on dynamic mazes compared to static, going from 34.4% to 46.9% for Alpha-Beta and Minimax, and from 15.6% to 28.1% for Hill Climbing. Beam Search gained 9.4 percentage points, moving from 34.4% to 43.8%. Expectimax and LRTA* each gained around 6 percentage points, with Expectimax going from 37.5% to 43.8% and LRTA* from 34.4% to 40.6%.

In fog of war mode, dynamic walls also helped rather than hurt. Online LRTA* showed the largest gain, jumping from 21.9% on static mazes to 43.8% on dynamic mazes, an improvement of 21.9 percentage points at every radius level. Online A* gained 11.5 percentage points overall, going from 37.5% on static to 49.0% on dynamic. The Exploration agent also improved on dynamic mazes, moving from 29.2% on static to 33.3% on dynamic, a gain of 4.1 percentage points.

4. Conclusions

4.1 What We Found

A few clear patterns came out of our experiments. When the agent can see everything, the game tree methods (Minimax, Alpha-Beta, Expectimax) won the most games. Alpha-Beta is the best choice in that group because it matches the others on win rate but runs about twice as fast. LRTA* is the right pick if speed matters more than winning every possible game: it runs in under 0.1 milliseconds per step, handles shifting walls naturally, and still wins 37.5% of the time which is close to the game tree agents.

In fog of war mode, Online A* is the clear winner by win rate (43.2%), but Online LRTA* is worth considering when compute time is tight since it runs just as fast as full visibility LRTA* at one thirtieth the time of Online A*. The Exploration agent is best suited for mazes where you have no idea where the goal is and mapping the area first is more important than speed.

Hill Climbing was the weakest agent overall. Mazes are full of spots where every nearby cell looks farther from the goal by straight-line distance, even though the real path goes through them. When Hill Climbing gets stuck at one of these spots, it cannot escape, and the pursuer catches it. Beam Search avoids some of this by keeping multiple options open, but it can still get trapped.

4.2 Limitations

The scoring function used inside the game tree search estimates distance using straight-line (Manhattan) distance, which does not account for walls. In a dense maze, the real walking distance can be much longer than the straight-line distance suggests. A more accurate estimate using BFS would improve decision quality, but it would make each game tree search much slower and reduce how many moves ahead the agent can look. We deliberately chose the faster estimate knowing it is less accurate.

We only ran experiments on mazes sized 11x11 and 13x13 because larger mazes make the game tree algorithms slow very quickly. In a bigger maze, LRTA* and Online LRTA* would still be fast, but Minimax and Expectimax would become unusable at reasonable depth limits. We expect the overall ranking of the algorithms to hold at larger sizes, but the speed differences would become much more pronounced.

In the fog of war experiments, the pursuer always knew the full maze and exactly where the agent was. A more realistic setup would limit what the pursuer can see too, which would likely help all three fog of war agents.

4.3 What Could Come Next

There are several directions that could make this project stronger. First, a natural extension would be to integrate and evaluate IDA* as a full agent in the adversarial pipeline, comparing it with our existing A* based approaches. IDA* uses almost no memory compared to A* because it does not store a visited list, which means it could handle much larger mazes without slowing down. The interesting question would be whether the memory savings come at a win rate cost in an adversarial setting, where thinking fast matters as much as thinking thoroughly.

Second, replacing the hand-written scoring formula with a small neural network could give the game tree agents a better sense of distance through walls without the full cost of running BFS at every leaf. The network would take as input the current positions of the agent and pursuer, the local wall pattern around each, and the distance to the goal, and it would be trained on thousands of finished games where the outcome is already known. This kind of learned evaluation function

is used in competitive game AI and could meaningfully improve decision quality in dense mazes where the Manhattan estimate is most misleading.

Third, the codebase already has some support for multiple pursuers, and fully testing that mode would let us study how agents behave when being chased from two directions at once. A single pursuer can usually be outmaneuvered by moving perpendicular to its path, but two pursuers working from opposite sides of the maze remove that option entirely, which would likely expose weaknesses in agents that rely on a single escape direction.

Fourth, the Exploration agent could be improved by switching to the navigation phase after mapping a certain percentage of the maze rather than waiting until it sees the goal directly. A threshold of around 60% coverage would likely capture most of the useful map structure while cutting the mapping phase short enough to reduce the time the pursuer has to close in. This change could be implemented with a single additional condition in the phase-switch logic and would be worth testing across the same range of maze sizes and densities used in the current experiments.

Works Cited

[1] P. Yap, "Grid-Based Path-Finding," in *Advances in Artificial Intelligence (AI 2002)*, R. Cohen and B. Spencer, Eds., Lecture Notes in Artificial Intelligence, vol. 2338. Berlin, Heidelberg: Springer, 2002, pp. 44–55.